45. Побитовые операторы

Побитовые операторы манипулируют отдельными битами в пределах переменной.

> ***Примечание:*** Для некоторых этот материал может показаться сложным. Если вы застряли или что-то не понятно — пропустите этот урок (и следующий), в будущем сможете вернуться и разобраться детально. Он не столь важен для прогресса в изучении языка C++, как другие уроки, и изложен здесь в большей мере для общего развития.

Оглавление:

1. Зачем нужны побитовые операторы?
2. Побитовый сдвиг влево (<<) и побитовый сдвиг вправо (>>)
3. Что!? Разве операторы << и >> используются не для вывода и ввода данных?
4. Побитовый оператор НЕ
5. Побитовые операторы И, ИЛИ и исключающее ИЛИ (XOR)
6. Побитовые операторы присваивания
7. Заключение
8. Тест
9. Ответы

Зачем нужны побитовые операторы?

В далеком прошлом компьютерной памяти было очень мало и ее сильно дорожали. Это было стимулом максимально разумно использовать каждый доступный бит. Например, в логическом типе данных bool есть всего лишь два возможных значения (true и false), которые могут быть представлены одним битом, но по факту занимают целый байт памяти! А это, в свою очередь, из-за того, что переменные используют уникальные адреса памяти, а они выделяется только в байтах. Переменная bool занимает 1 бит, а другие 7 бит — тратятся впустую.

Используя побитовые операторы, можно создавать функции, которые позволят уместить 8 значений типа bool в переменную размером 1 байт, что значительно сэкономит потребление памяти. В прошлом такой трюк был очень популярен. Но сегодня, по крайней мере, в прикладном программировании, это не так.

Теперь памяти стало существенно больше и программисты обнаружили, что лучше писать код так, чтобы было проще и понятнее его поддерживать, нежели усложнить его ради незначительной экономии памяти. Поэтому спрос на использование побитовых операторов несколько уменьшился, за исключением случаев, когда необходима уж максимальная оптимизация (например, в игровые программы, которые используют огромное количество данных; игры, где манипуляции с битами могут быть использованы для дополнительной скорости; встроенные программы, где память по-прежнему ограничена).

В языке C++ есть 6 побитовых операторов:

Оператор	Символ	Пример	Операция
Побитовый сдвиг влево	<<	x << y	Все биты в x смещаются влево на y бит
Побитовый сдвиг вправо	>>	x >> y	Все биты в x смещаются вправо на y бит
Побитовое НЕ	~	~x	Все биты в x меняются на противоположные
Побитовое И	&	x & y	Каждый бит в x И каждый соответствующий ему бит в y
Побитовое ИЛИ	|	x | y	Каждый бит в x ИЛИ каждый соответствующий ему бит в y
Побитовое исключающее ИЛИ (XOR)	^	x ^ y	Каждый бит в x XOR с каждым соответствующим ему битом в y

В побитовых операциях следует использовать только целочисленные типы данных unsigned, так как C++ не всегда гарантирует корректную работу побитовых операторов с целочисленными типами signed.

Правило: При работе с побитовыми операторами используйте целочисленные типы данных unsigned.

Побитовый сдвиг влево (<<) и побитовый сдвиг вправо (>>)

В языке C++ количество используемых бит основывается на размере типа данных (в 1 байте находится 8 бит). Оператор побитового сдвига влево (`<<`) сдвигает биты влево. Левый операнд является выражением, в котором он сдвигается, а правый — количество мест, на которое нужно сдвинуть. Поэтому в выражении `3 << 1` мы имеем в виду «сдвинуть биты влево в литерале 3 на одно место».

> ***Примечание:*** В следующих примерах мы будем работать с 4-битными двоичными значениями.

Рассмотрим число 3, которое в двоичной системе равно 0011:

```
3 = 0011
3 << 1 = 0110 = 6
3 << 2 = 1100 = 12
3 << 3 = 1000 = 8
```

В последнем третьем случае один бит перемещается за пределы самого литерала! Биты, сдвинутые за пределы двоичного числа, теряются навсегда.

Оператор побитового сдвига вправо (`>>`) сдвигает биты вправо. Например:

```
12 = 1100
12 >> 1 = 0110 = 6
12 >> 2 = 0011 = 3
12 >> 3 = 0001 = 1
```

В третьем случае мы снова переместили бит за пределы литерала. Он также потерялся навсегда.

Хотя в примерах, приведенных выше, мы смещаем биты только в литералах, мы также можем смещать биты и в переменных:

```
x = unsigned int x = 4;
x << 1;
```

Следует помнить, что результаты операций с побитовыми сдвигами в разных компиляторах могут отличаться.

Что!? Разве операторы << и >> используются не для вывода и ввода данных?

И для этого тоже.

Сейчас польза от использования побитовых операторов не так велика, как это было раньше. Сейчас в большинстве случаев оператор побитового сдвига влево используется для вывода данных. Например, рассмотрим следующую программу:

```
#include <iostream>
using namespace std;

int main()
{
    unsigned int x = 4;
    x = x << 1; // сначала мы используем для побитового сдвига влево
    std::cout << x; // оператор << используется для вывода данных в консоль.

    return 0;
}
```

Результат выполнения программы:

```
8
```

А как компьютер понимает, когда нужно применить оператор побитового сдвига влево, а когда выводить данные? Всё очень просто. std::cout **перегружает** значения оператора `<<` по умолчанию на ввод (вывод данных в консоль). Когда компилятор видит, что левым операндом оператора `<<` является std::cout, то он понимает, что должен произойти вывод данных. Если левым операндом является переменная целочисленного типа данных, то компилятор понимает, что должен произойти побитовый сдвиг влево (операция по умолчанию).

Побитовый оператор НЕ

Побитовый оператор НЕ (`~`), пожалуй, самый простой для объяснения и понимания. Он просто меняет каждый бит на противоположный, например, с 0 на 1 или с 1 на 0. Обратите внимание, результаты побитового НЕ зависят от размера типа данных!

Предположим, что размер типа данных составляет 4 бита:

```
4 = 0100
~ 4 = 1011 (двоичное) = 11 (десятичное)
```

Предположим, что размер типа данных составляет 8 бит:

```
4 = 0000 0100
~ 4 = 1111 1011 (двоичное) = 251 (десятичное)
```

Побитовые операторы И, ИЛИ и исключающее ИЛИ (XOR)

Побитовые операторы И (`&`) и ИЛИ (`|`) работают аналогично логическим операторам И и ИЛИ. Однако, побитовые операторы применяются к каждому биту отдельно! Например, рассмотрим выражение `5 | 6`. В двоичной системе это `0101 | 0110`. В любой побитовой операции операнды лучше всего размещать следующим образом:

```
0 1 0 1 // 5
0 1 1 0 // 6
```

А затем применить операцию к каждому столбцу с битами по отдельности. Как вы помните, логическое ИЛИ возвращает true (1), если один из двух или оба операнды истинны (1). Аналогичным образом работает и **побитовое ИЛИ**. Выражение `5 | 6` обрабатывается следующим образом:

```
0 1 0 1 // 5
0 1 1 0 // 6
--------
0 1 1 1 // 7
```

Результат:

```
0111 (двоичное) = 7 (десятичное)
```

Также можно обрабатывать и комплексные выражения ИЛИ, например, `1 | 4 | 6`. Если хоть один бит в столбце равен 1, то результат целого столбца — 1. Например:

```
0 0 0 1 // 1
0 1 0 0 // 4
0 1 1 0 // 6
--------
0 1 1 1 // 7
```

Результатом `1 | 4 | 6` является десятичное `7`.

Побитовое И работает аналогично логическому И — возвращается true, только если оба бита в столбце равны 1. Рассмотрим выражение `5 & 6`:

```
0 1 0 1 // 5
0 1 1 0 // 6
--------
0 1 0 0 // 4
```

Также можно решать и комплексные выражения И, например, `1 & 3 & 7`. Только при условии, что все биты в столбце равны 1, результатом столбца будет 1:

```
0 0 0 1 // 1
0 0 1 1 // 3
0 1 1 1 // 7
--------
0 0 0 1 // 1
```

Последний оператор — **побитовое исключающее ИЛИ** (`^`) (englishe. **«XOR»** от англ. «eXclusive **OR**»). При обработке двух операндов, исключающее ИЛИ возвращает true (1), только если один и только один из операндов является истинным (1). Если таких нет или все операнды равны 1, то результатом будет false (0). Рассмотрим выражение `6 ^ 3`:

```
0 1 1 0 // 6
0 0 1 1 // 3
--------
0 1 0 1 // 5
```

Также можно решать и комплексные выражения XOR, например, `1 ^ 3 ^ 7`. Если единиц в столбце чётное количество, то результатом будет 0, если же нечётное количество, то результат — 1. Например:

```
0 0 0 1 // 1
0 0 1 1 // 3
0 1 1 1 // 7
--------
0 1 0 1 // 5
```

Побитовые операторы присваивания

Как и в случае с арифметическими операторами присваивания, язык C++ предоставляет побитовые операторы присваивания для облегчения внесения изменений в переменные.

Оператор	Символ	Пример	Операция
Присваивание с побитовым сдвигом влево	<<=	x <<= y	Сдвигаем биты в x влево на y бит
Присваивание с побитовым сдвигом вправо	>>=	x >>= y	Сдвигаем биты в x вправо на y бит
Присваивание с побитовой операцией ИЛИ	|=	x |= y	Присваивание результата выражения x | y переменной x
Присваивание с побитовой операцией И	&=	x &= y	Присваивание результата выражения x & y переменной x
Присваивание с побитовой операцией исключающего ИЛИ	^=	x ^= y	Присваивание результата выражения x ^ y переменной x

Например, вместо `x = x << 1;` мы можем написать `x <<= 1;`.

Заключение

При работе с побитовыми операторами (используя метод столбца) не забывайте о том, что:

- При вычислении побитового ИЛИ, если хоть один из битов в столбце равен 1, то результат целого столбца равен 1.
- При вычислении побитового И, если все биты в столбце равны 1, то результат целого столбца равен 1.
- При вычислении побитового исключающего ИЛИ (XOR), если единиц в столбце нечётное количество, то результат равен 1.

Тест

Задание №1

Какой результат `0110 >> 2` в двоичной системе счисления?

Задание №2

Какой результат `5 | 12` в десятичной системе счисления?

Задание №3

Какой результат `5 & 12` в десятичной системе счисления?

Задание №4

Какой результат `5 ^ 12` в десятичной системе счисления?

Ответы

Ответ №1

Результатом `0110 >> 2` является двоичное число `0001`.

Ответ №2

Выражение `5 | 12`:

```
0 1 0 1
1 1 0 0
--------
1 1 0 1 // 13 (десятичное)
```

Ответ №3

Выражение `5 & 12`:

```
0 1 0 1
1 1 0 0
--------
0 1 0 0 // 4 (десятичное)
```

Ответ №4

Выражение `5 ^ 12`:

```
0 1 0 1
1 1 0 0
--------
1 0 0 1 // 9 (десятичное)
```